

TSFM-Lens: A Python Toolkit for Controlled, Causal and Confirmatory Interpretability of Time-Series Foundation Models

Dan Kushnir

DAN.KUSHNIR@SPIALERT.COM

Abstract

TSFM-LENS is an open-source Python toolkit that turns a list of time-series foundation model (TSFM) checkpoints into a single self-contained interpretability report with one command. It compares models with different architectures on a measured common time axis, validates every probe on a synthetic benchmark with exact generative ground truth, defines concepts by their causal effect on the forecast against matched random nulls, and tests exploratory claims once on a sealed held-out split. Every reported number carries an evidence class and a plain-language explanation. The toolkit ships adapters for TimesFM, the Chronos family, Sundial, Timer and Lag-Llama, a zero-code adapter for Hugging Face checkpoints, a complete example run and a test suite of over two thousand tests. It is released under the MIT license at <https://github.com/DanK10097110/TSFM-Interp>.

Keywords: interpretability, time-series foundation models, sparse autoencoders, causal intervention, open-source software

1 Introduction

Pretrained forecasters such as TimesFM (Das et al., 2024), Chronos (Ansari et al., 2024, 2025), Sundial (Liu et al., 2025) and Timer (Liu et al., 2024) are used zero-shot, yet little is known about what they compute. Libraries such as TransformerLens (Nanda and Bloom, 2022), pyvene (Wu et al., 2024), mnsight (Fiotto-Kaufman et al., 2025) and SAELens (Bloom et al., 2024) provide hooks, patching and sparse dictionaries for one model at a time. Three properties of TSFMs make a direct transfer unreliable: tokens cover 1 to 96 time steps depending on the architecture, so positions are not comparable across models; real series carry no labels for trend, seasonality or noise, so probes cannot be checked; and the causal effects of internal features are small enough that an uncontrolled measurement yields a confident false positive. TSFM-LENS is built around these problems. It is a *comparative* tool: its unit of analysis is a panel of models on a shared benchmark, and its output is a report that a non-specialist can read and an expert can audit.

2 Architecture

The repository contains two separately installable packages (Figure 1). `tsfm_benchmark` builds and validates corpora; `tsfm_lens` analyzes models and renders the report. Their dependencies overlap only in NumPy, SciPy and PyYAML, so the benchmark can be used on its own.

Benchmark. Generators come in two labeled tiers: a *synthetic* tier (additive parametric series and frozen archetypes) that touches no real data and records every generative

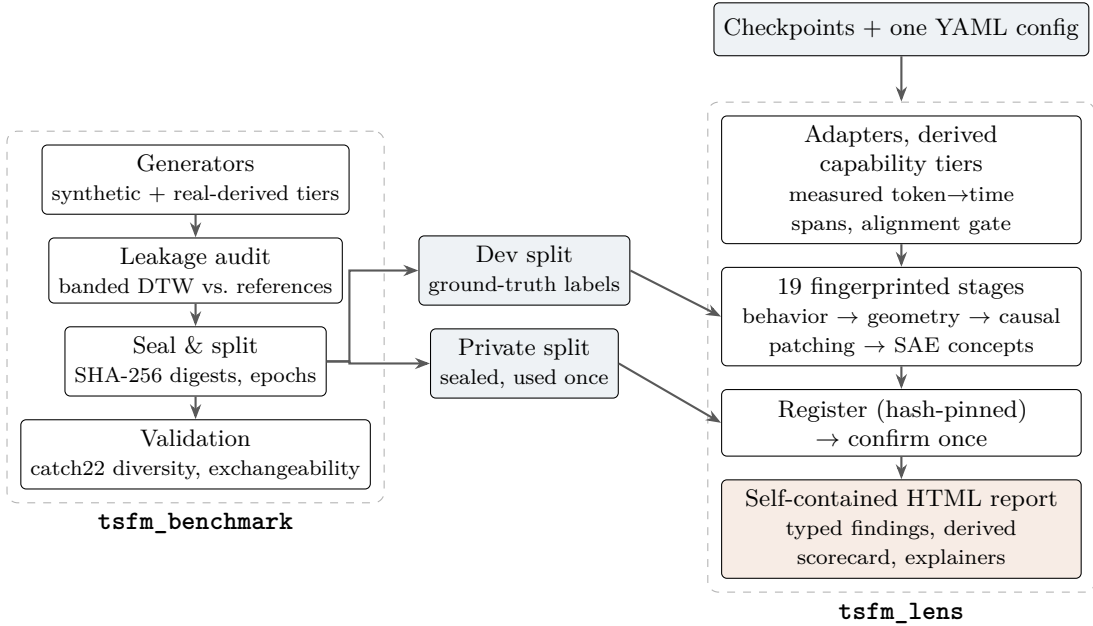


Figure 1: Architecture of TSFM-LENS. The benchmark package produces leakage-audited, sealed corpora with exact ground truth. The analysis package loads each checkpoint through an adapter, runs a gated stage pipeline on the dev split, confirms registered claims once on the private split and renders one report.

component, and a *real-derived* tier (mixtures, block bootstrap, sequential synthesis) with attributional ground truth. Every sample passes a leakage audit against real reference archives; dev and private splits come from disjoint seed ranges, are hashed, and can be re-minted as fresh epochs. A validation module measures diversity in catch22 feature space and tests dev/private exchangeability. Structured corruptions and single-knob generator counterfactuals provide controlled perturbations with known effects.

Adapters. A model is wrapped by an adapter that exposes loading, prediction, the module and token time spans. Its capability tier (black box, observable, steerable, decomposable) is *derived* from the methods it implements, and stages that need a higher tier are skipped with the reason rendered in the report. A zero-code adapter discovers layers and *measures* token spans from impulse responses for Hugging Face checkpoints, refusing rather than guessing; hand-written adapters start from a template, and a single command (`--check-adapter`) runs conformance, reach and alignment checks.

Alignment and stages. Tokens are overlap-pooled onto common time windows, so every cross-model statistic compares the same stretch of time; an alignment gate verifies this per layer during extraction. The pipeline has 19 stages grouped by the evidence they can produce: behavioral (accuracy, calibration, cost, tokenizer checks), descriptive (effective dimension, logit-lens style depth profiles), geometric and linear-translatable (CKA, RSA, stitching gain over an input baseline), causal within-model (corruption fingerprints, activation patching,

head ablation) and concept-level (TopK sparse autoencoders, a feature ablation battery, cross-model concept matching). Each stage declares the configuration keys it depends on; these are fingerprinted, so cached artifacts are reused only when still valid.

Causal concepts. A sparse-dictionary feature counts as causal only if removing it changes a forecast channel (level, trend, seasonality, shape, accuracy, ...) more than removing a random direction of matched size on the same series, after reach controls confirm that the intervention lands (self-patch exactly zero, cross-patch nonzero). Causal features are clustered into concepts within and across models, and sharing is tested at two levels: whether two models’ concepts select the *same inputs*, and whether they have the *same effect on those inputs*, each against its own null.

Statistics and confirmation. All intervals and p -values resample series rather than windows. Every test belongs to a named correction family, and a multiplicity ledger flags families whose smallest attainable adjusted p -value cannot reach the significance level. Exploratory claims are frozen in a hash-pinned registry and tested once on the sealed split; a repeated look is recorded.

Report. The output is one HTML file without external dependencies. Sections pose plain questions (“Is the comparison fair?”, “What causally drives the forecast?”), every figure carries a caption and an expandable explanation of how to read it and what it cannot show, the scorecard is derived from artifacts by explicit rules with no model names hard-coded, and every claim is a typed record with an evidence class and a generated caveat. Skipped or failed sections render visibly with their reason.

3 Usage

After `pip install -e .` && `pip install -e tsfm_lens`, a full analysis is

```
python run.py --config configs/examples/large_4model.yaml
```

Presets cover a CPU-only smoke test with mock models (minutes), a single model and a pair. The CLI also offers a preflight check (`--doctor`), partial reruns (`--stages`, `--force`), alignment and adapter checks for new checkpoints, and a provenance diff between a run’s recorded environment and the current one. New report sections, claims and configuration fields plug into declared interfaces that keep the fingerprints and evidence classes consistent.

Example. The repository includes a complete run on TimesFM 2.5, Chronos-2, Sundial and Chronos-Bolt (`examples/concept_atlas_v2`: report, configuration and per-stage artifacts; 6.4 hours on one 24 GB GPU). In it, 293 sparse features clear their causal nulls, mostly acting on forecast level and shape; the models agree on which inputs a concept groups (20 of 20 registered claims confirmed on 800 sealed series) but rarely on what it does to them. A curated ledger (`FINDINGS.md`) records every result, positive and negative, with its evidence class and reproduction command, including cases where a built-in control overturned a plausible but wrong result.

4 Quality and Availability

The code is released under the MIT license. The test suite (over 2,100 tests for `tsfm_lens`, plus golden-hash regeneration tests for the benchmark) runs in continuous integration on mock adapters, against pinned dependencies and, weekly, against the latest releases of the load-bearing libraries. Tests follow a planted-answer standard: each load-bearing assertion must be shown to fail against a deliberately planted regression. Documentation includes a README, a guide to adding models with a pitfall checklist, a worked report walkthrough, and stage, adapter and glossary documentation generated from the code (with a `--check` mode that detects drift). Exact dependency pins and the reason for each are recorded in `DEPENDENCIES.md`, and every run records its environment. TSFM-LENS is univariate by design, captures only the parts of a model an adapter exposes, and relies on synthetic ground truth whose generators encode assumptions about which structure matters; the report states these limits alongside the results they qualify.

References

- A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. Pineda Arango, S. Kapoor, J. Zschiegner, D. C. Maddix, H. Wang, M. W. Mahoney, K. Torkkola, A. G. Wilson, M. Bohlke-Schneider, and Y. Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024.
- A. F. Ansari, O. Shchur, et al. Chronos-2: From univariate to universal forecasting. *arXiv preprint arXiv:2510.15821*, 2025.
- J. Bloom, C. Tigges, A. Duong, and D. Chanin. SAELens. <https://github.com/jbloomAus/SAELens>, 2024.
- A. Das, W. Kong, R. Sen, and Y. Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 2024.
- J. Fiotto-Kaufman, A. R. Loftus, E. Todd, J. Brinkmann, K. Pal, D. Troitskii, M. Ripa, A. Belfki, C. Rager, C. Juang, et al. NNsight and NDIF: Democratizing access to open-weight foundation model internals. In *International Conference on Learning Representations*, 2025.
- Y. Liu, H. Zhang, C. Li, X. Huang, J. Wang, and M. Long. Timer: Generative pre-trained transformers are large time series models. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Y. Liu, G. Qin, Z. Shi, Z. Chen, C. Yang, X. Huang, J. Wang, and M. Long. Sundial: A family of highly capable time series foundation models. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. arXiv:2502.00816.
- N. Nanda and J. Bloom. TransformerLens. <https://github.com/TransformerLensOrg/TransformerLens>, 2022.

Z. Wu, A. Geiger, A. Arora, J. Huang, Z. Wang, N. D. Goodman, C. D. Manning, and C. Potts. pyvene: A library for understanding and improving PyTorch models via interventions. In *Proceedings of NAACL-HLT (System Demonstrations)*, 2024.